Line-by-Line Code Explanation

`finegym.ipynb`

V-JEPA 2 vs Qwen2.5-VL

FineGym — Gymnastics Event Classification

4 Classes: Floor Exercise (FX), Balance Beam (BB),
Uneven Bars (UB), Vault (VT)

Thesis: *Comparative Study of Vision-Language Models:*
VL-JEPA versus Traditional Token-Based VLMs

Author: Erize Rai

Roll No: 8025006

June 20, 2026

Contents

1 Overview **2**

2 Cell 1: Package Installation **3**

3 Cell 2: Imports and Configuration **4**

4 Cell 3: Download FineGym Annotations **9**

5 Cell 4: Download Video Clips and Build Dataset **11**

6 Cell 5: V-JEPA 2 Feature Extraction **14**

7 Cell 6: Qwen2.5-VL Feature Extraction **18**

8 Cell 7: Load Features and Create Train/Eval Split **20**

9 Cell 8: Run Classifiers and Compare **21**

10 Cell 9: Final Comparison Table **23**

Overview

This document provides a detailed line-by-line explanation of every code cell in the `finegym.ipynb` Jupyter Notebook. The notebook benchmarks two vision models—**V-JEPA 2** (654M parameters, self-supervised video encoder by Meta) and **Qwen2.5-VL** (7B parameters, autoregressive vision-language model by Alibaba)—on the **FineGym** dataset for gymnastics event classification (4 classes).

Experimental setup:

- **Dataset**: FineGym — gymnastics event classification
- **Classes**: Balance Beam, Floor Exercise, Uneven Bars, Vault
- **Method**: Frozen backbone features + traditional ML classifiers (no fine-tuning)
- **Platform**: Kaggle (GPU P100 or T4 x2)
- **Key Result**: V-JEPA 2 achieved 97.1% accuracy (F1: 96.4%) vs Qwen2.5-VL at 80.0% accuracy (F1: 79.7%)

Cell 1: Package Installation

```
!pip install -q transformers accelerate bitsandbytes peft
    datasets huggingface_hub opencv-python-headless
    scikit-learn yt-dlp
```

Explanation:

- `!` — The exclamation mark runs a shell command from within a Jupyter notebook cell.
- `pip install -q` — Installs Python packages using pip. The `-q` (quiet) flag suppresses verbose output, showing only errors and warnings.
- **Packages installed:**
  - `transformers` — Hugging Face library providing pre-trained model architectures (V-JEPA 2, Qwen2.5-VL) and their processors.
  - `accelerate` — Hugging Face library for efficient model loading and distributed inference, enabling device placement and mixed-precision.
  - `bitsandbytes` — Library for 4-bit and 8-bit quantization of neural network weights, used to fit the 7B-parameter Qwen model into GPU memory.
  - `peft` — Parameter-Efficient Fine-Tuning library (imported as a dependency but not directly used for training in this notebook).
  - `datasets` — Hugging Face datasets library for loading and managing datasets (available as a utility).
  - `huggingface_hub` — Client library for downloading models and datasets from the Hugging Face Hub.
  - `opencv-python-headless` — OpenCV without GUI dependencies, used for video reading and frame extraction.
  - `scikit-learn` — Machine learning library providing classifiers (SVM, Random Forest, MLP, etc.) and evaluation metrics.
  - `yt-dlp` — YouTube video downloader (fork of youtube-dl), used to download FineGym clips from YouTube.

Output: Installation progress bars and a dependency conflict warning (non-critical, related to pre-installed Kaggle packages).

Cell 2: Imports and Configuration

```
import torch, sys, os, time, json, gc, random, tempfile,
       subprocess, urllib.request
import numpy as np
import torch.nn as nn
from pathlib import Path
from collections import Counter
from PIL import Image
```

Explanation:

- `torch` — PyTorch deep learning framework; used for model inference, tensor operations, and GPU management.
- `sys` — System-specific parameters (imported for potential use).
- `os` — Operating system interface; used for file/directory operations (`os.path.exists`, `os.makedirs`, etc.).
- `time` — Time measurement; used for tracking extraction speed and ETA calculations.
- `json` — JSON parsing/serialization; used for reading FineGym annotations and saving results.
- `gc` — Garbage collector; explicitly called to free memory after unloading models.
- `random` — Random number generator; used for shuffling clips and reproducibility.
- `tempfile` — Temporary file creation (imported for potential use).
- `subprocess` — Process execution; used to call `yt-dlp` as a shell command for downloading videos.
- `urllib.request` — HTTP request library; used to download the FineGym annotation JSON file.
- `numpy` (as `np`) — Numerical computing library; used for array operations, linear indexing, and data manipulation.
- `torch.nn` (as `nn`) — PyTorch's neural network module (imported for potential custom layers).
- `Path` — Object-oriented filesystem path handling.
- `Counter` — Dictionary subclass for counting hashable objects; used to count clips per class.
- `Image` (from PIL) — Python Imaging Library; used to create PIL Image objects from frame arrays.

```
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
```

```
    matthews_corrcoef, cohen_kappa_score,
    classification_report, confusion_matrix,
    top_k_accuracy_score,
)
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier,
                             StackingClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.decomposition import PCA
import cv2
```

Explanation:

Scikit-learn evaluation metrics:

- `accuracy_score` — Fraction of correctly classified samples.
- `precision_score` — Ratio of true positives to predicted positives; measures exactness.
- `recall_score` — Ratio of true positives to actual positives; measures completeness.
- `f1_score` — Harmonic mean of precision and recall; balances both metrics.
- `matthews_corrcoef` (MCC) — Correlation coefficient between observed and predicted classifications; ranges from -1 to $+1$.
- `cohen_kappa_score` — Measures inter-rater agreement, accounting for chance agreement.
- `classification_report` — Generates a text summary of precision, recall, and F1 per class.
- `confusion_matrix` — Computes the C_{ij} matrix where entry (i, j) counts samples with true label i predicted as j.
- `top_k_accuracy_score` — Checks whether the true label is among the top-k predictions.

Preprocessing and classifiers:

- `StandardScaler` — Standardizes features by removing the mean and scaling to unit variance ($z = \frac{x-\mu}{\sigma}$).
- `SVC` — Support Vector Classifier with RBF (Radial Basis Function) kernel.
- `RandomForestClassifier` — Ensemble of decision trees using bagging.
- `StackingClassifier` — Combines multiple classifiers via a meta-learner (im-

ported but not used).

- `MLPClassifier` — Multi-Layer Perceptron; a feedforward neural network classifier.
- `LogisticRegression` — Linear classifier using logistic/softmax function.
- `KNeighborsClassifier` — k-Nearest Neighbors; classifies based on majority vote of k nearest training points.
- `PCA` — Principal Component Analysis; reduces feature dimensionality while retaining 95% of variance.
- `cv2` — OpenCV; used for video capture (`VideoCapture`), frame reading, color conversion, and resizing.

```
device = torch.device('cuda' if torch.cuda.is_available()
                      else 'cpu')
if device.type == 'cuda':
    gpu = torch.cuda.get_device_name(0)
    props = torch.cuda.get_device_properties(0)
    mem = getattr(props, 'total_memory', None) or
          getattr(props, 'total_mem', 0)
    print(f'GPU: {gpu} ({mem/1e9:.1f} GB)')
else:
    print('WARNING: No GPU!')
```

Explanation:

- `torch.device(...)` — Selects CUDA GPU if available; otherwise falls back to CPU.
- `torch.cuda.is_available()` — Returns `True` if a CUDA-capable GPU is detected.
- `torch.cuda.get_device_name(0)` — Returns the name of GPU device 0 (e.g., "Tesla T4").
- `torch.cuda.get_device_properties(0)` — Returns a struct with GPU properties including total memory.
- `getattr(props, 'total_memory', None)` — Safely retrieves the `total_memory` attribute (API varies across PyTorch versions).
- The print statement shows: `GPU: Tesla T4 (15.6 GB)`.

```
SEED = 42
NUM_FRAMES = 8
IMG_SIZE = 224
TRAIN_SPLIT = 0.8
```

```
CKPT_EVERY = 100
SAVE_DIR = '/kaggle/working'
CLIPS_DIR = os.path.join(SAVE_DIR, 'finegym_clips')
os.makedirs(CLIPS_DIR, exist_ok=True)
```

Explanation:

Hyperparameters and configuration constants:

- `SEED = 42` — Random seed for reproducibility across all random operations.
- `NUM_FRAMES = 8` — Number of uniformly-sampled frames extracted per video clip for V-JEPA 2.
- `IMG_SIZE = 224` — Target spatial resolution (224×224 pixels); standard for vision transformers.
- `TRAIN_SPLIT = 0.8` — 80% of data for training, 20% for evaluation.
- `CKPT_EVERY = 100` — Save a checkpoint every 100 clips to enable resuming if interrupted.
- `SAVE_DIR` — Kaggle's persistent output directory; files here survive kernel restarts.
- `CLIPS_DIR` — Subdirectory for storing downloaded video clips.
- `os.makedirs(..., exist_ok=True)` — Creates the directory tree; does not raise an error if it already exists.

```
EVENT_NAMES = {1: 'Balance_Beam', 2: 'Uneven_Bars',
               3: 'Floor_Exercise', 4: 'Vault'}
EVENT_LIST = ['Balance_Beam', 'Floor_Exercise',
              'Uneven_Bars', 'Vault']
NUM_CLASSES = 4

torch.manual_seed(SEED)
np.random.seed(SEED)
random.seed(SEED)
print(f'Classes: {EVENT_LIST}')
```

Explanation:

- `EVENT_NAMES` — Maps FineGym's 1-indexed event IDs to human-readable names.
- `EVENT_LIST` — Sorted list of class names; the index in this list becomes the integer label (0–3).
- `NUM_CLASSES = 4` — Total number of classification categories.
- `torch.manual_seed(SEED)` — Seeds PyTorch's random number generator for reproducible tensor operations.

- `np.random.seed(SEED)` — Seeds NumPy's random number generator.
- `random.seed(SEED)` — Seeds Python's built-in random module.
- Setting all three seeds ensures deterministic behavior across the entire pipeline.

Output: `Classes: ['Balance_Beam', 'Floor_Exercise', 'Uneven_Bars', 'Vault']`

Cell 3: Download FineGym Annotations

```
ANNO_URL = 'https://sdolivia.github.io/FineGym/resources/
            dataset/finegym_annotation_info_v1.1.json'
ANNO_PATH = os.path.join(SAVE_DIR,
                         'finegym_annotations.json')

if not os.path.exists(ANNO_PATH):
    print('Downloading FineGym annotations...')
    urllib.request.urlretrieve(ANNO_URL, ANNO_PATH)
    print('Done')

with open(ANNO_PATH) as f:
    annotations = json.load(f)
```

Explanation:

- `ANNO_URL` — URL to the official FineGym v1.1 annotation JSON hosted on the FineGym project page.
- `ANNO_PATH` — Local path where the downloaded annotation file will be saved.
- `os.path.exists(ANNO_PATH)` — Checks if annotations were already downloaded (skip if so).
- `urllib.request.urlretrieve(URL, PATH)` — Downloads the file from the URL and saves it to the local path.
- `json.load(f)` — Parses the JSON file into a Python dictionary. The structure is: {`youtube_id: {event_key: {event: int, timestamps: [[start, end]], ...}}`}.

```
all_clips = []
for yt_id, events in annotations.items():
    for event_key, info in events.items():
        event_id = info['event']
        if event_id not in EVENT_NAMES:
            continue
        ts = info['timestamps'][0]
        all_clips.append({
            'yt_id': yt_id,
            'event_key': event_key,
            'event_id': event_id,
            'event_name': EVENT_NAMES[event_id],
            'start': ts[0],
```

```
            'end': ts[1],
        })

print(f'Total event clips: {len(all_clips)}')
counts = Counter(c['event_name'] for c in all_clips)
for name in sorted(counts):
    print(f'  {name:<20} {counts[name]:>5}')
```

Explanation:

- `all_clips = []` — Initializes an empty list to hold parsed clip metadata.
- **Outer loop** iterates over each YouTube video ID in the annotations.
- **Inner loop** iterates over each annotated event within that video.
- `info['event']` — The event type ID (1=BB, 2=UB, 3=FX, 4=VT).
- `if event_id not in EVENT_NAMES: continue` — Skips non-target events (e.g., sub-actions).
- `info['timestamps'][0]` — Extracts the first timestamp pair `[start_sec, end_sec]`.
- Each clip's metadata (YouTube ID, event key, label, timestamps) is appended to `all_clips`.
- `Counter` tallies clips per class for a distribution summary.

Output: 6,915 total clips — Balance Beam: 2,436 — Floor Exercise: 1,597 — Uneven Bars: 1,214 — Vault: 1,668.

Cell 4: Download Video Clips and Build Dataset

```
def download_clip(yt_id, start, end, out_path,
                  max_retries=2):
    if os.path.exists(out_path) and
       os.path.getsize(out_path) > 1000:
        return True
    duration = end - start
    url = f'https://www.youtube.com/watch?v={yt_id}'
    for attempt in range(max_retries):
        try:
            cmd = [
                'yt-dlp', '--quiet', '--no-warnings',
                '--download-sections', f'*{start}-{end}',
                '-f', 'worst[ext=mp4]/worst',
                '--force-keyframes-at-cuts',
                '-o', out_path, url
            ]
            result = subprocess.run(cmd,
                capture_output=True, text=True,
                timeout=120)
            if os.path.exists(out_path) and
               os.path.getsize(out_path) > 500:
                return True
        except (subprocess.TimeoutExpired, Exception):
            pass
    return False
```

Explanation:

- **Function purpose:** Downloads a specific time segment of a YouTube video using `yt-dlp`.
- **Skip check:** If the output file already exists and is larger than 1,000 bytes, the download is skipped (resume support).
- `--download-sections` — yt-dlp flag that downloads only the specified time range `*start-end`.
- `-f worst[ext=mp4]/worst` — Selects the lowest quality MP4 format (faster download, sufficient for frame extraction).
- `--force-keyframes-at-cuts` — Re-encodes at cut points for precise trimming.

- `subprocess.run(..., timeout=120)` — Runs the command with a 2-minute timeout to avoid hanging on unavailable videos.
- Returns `True` if the file was successfully downloaded (size $>$ 500 bytes), `False` otherwise.
- Retries up to `max_retries=2` times on failure.

```
def load_frames_from_file(video_path, n_frames=8,
                          size=224):
    cap = cv2.VideoCapture(video_path)
    if not cap.isOpened(): return None
    total = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    if total < 1: cap.release(); return None
    idxs = np.linspace(0, total - 1, n_frames, dtype=int)
    frames = []
    for i in idxs:
        cap.set(cv2.CAP_PROP_POS_FRAMES, i)
        ok, fr = cap.read()
        if not ok:
            fr = np.zeros((size, size, 3), dtype=np.uint8)
        fr = cv2.cvtColor(fr, cv2.COLOR_BGR2RGB)
        fr = cv2.resize(fr, (size, size))
        frames.append(Image.fromarray(fr))
    cap.release()
    return frames
```

Explanation:

- **Function purpose:** Extracts `n_frames` uniformly-spaced frames from a video file.
- `cv2.VideoCapture(video_path)` — Opens the video file for reading.
- `cap.isOpened()` — Verifies the video was successfully opened.
- `cv2.CAP_PROP_FRAME_COUNT` — Property to get total number of frames in the video.
- `np.linspace(0, total-1, n_frames, dtype=int)` — Computes `n_frames` evenly-spaced frame indices across the entire video duration.
- `cap.set(cv2.CAP_PROP_POS_FRAMES, i)` — Seeks to frame index `i`.
- `cap.read()` — Reads the frame at the current position; returns `(success_bool, frame_array)`.
- If reading fails, a black placeholder frame (all zeros) is used.
- `cv2.cvtColor(..., cv2.COLOR_BGR2RGB)` — Converts from OpenCV's default BGR to RGB color order.

- `cv2.resize(fr, (size, size))` — Resizes to 224×224 pixels.
- `Image.fromarray(fr)` — Converts the NumPy array to a PIL Image (required by model processors).
- Returns a list of `n_frames` PIL Image objects.

```
MAX_PER_CLASS = 300
sampled_clips = []
for event_name in EVENT_LIST:
    cls_clips = [c for c in all_clips
                 if c['event_name'] == event_name]
    random.shuffle(cls_clips)
    sampled_clips.extend(cls_clips[:MAX_PER_CLASS])
random.shuffle(sampled_clips)
```

Explanation:

- `MAX_PER_CLASS = 300` — Limits to 300 clips per class for manageability (1,200 total).
- For each class, all matching clips are collected, shuffled, and the first 300 are selected.
- The combined list is shuffled again to ensure random ordering during download and processing.
- This balanced sampling prevents class imbalance during feature extraction.

The remainder of Cell 4 contains the download loop with:

- **Resume support** via a JSON log file (`dl_status`) tracking attempted downloads.
- **Progress reporting** every 20 clips showing success/fail counts and elapsed time.
- **Final clip assembly**: Only successfully downloaded files (> 500 bytes) are kept with their integer labels.

Output: 172 clips successfully downloaded (many YouTube videos are no longer available), distributed as: Balance Beam: 47, Floor Exercise: 45, Uneven Bars: 47, Vault: 33.

Cell 5: V-JEPA 2 Feature Extraction

```
from transformers import VJEPA2Model,
                         VJEPA2VideoProcessor

VJEPA_FEAT = os.path.join(SAVE_DIR,
                          'vjepa2_finegym_features.pt')
VJEPA_CKPT = os.path.join(SAVE_DIR,
                          'vjepa2_finegym_ckpt.pt')
```

Explanation:

- `VJEPA2Model` — The V-JEPA 2 vision transformer model class from Hugging Face. V-JEPA 2 is a self-supervised video encoder (654M parameters) that learns video representations by predicting masked spatiotemporal patches.
- `VJEPA2VideoProcessor` — Preprocessor that handles frame normalization, resizing, and conversion to model input tensors.
- `VJEPA_FEAT` — Path for the final saved features file.
- `VJEPA_CKPT` — Path for intermediate checkpoints (for crash recovery).

```
if os.path.exists(VJEPA_FEAT):
    print(f'V-JEPA features COMPLETE: {VJEPA_FEAT}')
    vdata = torch.load(VJEPA_FEAT, map_location='cpu',
                       weights_only=True)
else:
    print('Loading V-JEPA 2...')
    vproc = VJEPA2VideoProcessor.from_pretrained(
        'facebook/vjepa2-vith-fpc64-256')
    vmodel = VJEPA2Model.from_pretrained(
        'facebook/vjepa2-vith-fpc64-256',
        torch_dtype=torch.float16,
        attn_implementation='sdpa',
    ).to(device).eval()
```

Explanation:

- **Skip if done:** If the final features file exists, it is loaded directly (no re-extraction needed).
- `torch.load(..., weights_only=True)` — Loads only tensor data (security measure against arbitrary code execution in pickle files).

- `from_pretrained('facebook/vjepa2-vith-fpc64-256')` — Downloads and loads the pre-trained V-JEPA 2 ViT-Huge model:
  - `vith` — Vision Transformer Huge architecture.
  - `fpc64` — 64 frames per clip capacity.
  - `256` — 256-pixel input resolution support.
- `torch_dtype=torch.float16` — Uses half-precision (FP16) to reduce memory usage by 50%.
- `attn_implementation='sdpa'` — Uses PyTorch's Scaled Dot-Product Attention (faster, memory-efficient flash attention).
- `.to(device)` — Moves model weights to GPU.
- `.eval()` — Sets the model to evaluation mode (disables dropout, batch normalization updates).

```
    # Resume from checkpoint if available
    features, labels, start_idx = [], [], 0
    if os.path.exists(VJEPA_CKPT):
        ckpt = torch.load(VJEPA_CKPT, ...)
        features = list(ckpt['features'].unbind(0))
        labels = ckpt['labels'].tolist()
        start_idx = int(ckpt['next_idx'])
        del ckpt; gc.collect()
```

Explanation:

- **Checkpoint resume logic:** If a partial checkpoint exists (from a previous interrupted run), previously extracted features are loaded.
- `ckpt['features'].unbind(0)` — Splits the stacked feature tensor back into a list of individual feature vectors.
- `start_idx` — The index to resume processing from.
- `del ckpt; gc.collect()` — Deletes the checkpoint variable and forces garbage collection to free memory.

```
    for i in range(start_idx, total):
        clip = final_clips[i]
        frames = load_frames_from_file(clip['path'],
                     NUM_FRAMES, IMG_SIZE)
        if frames is None: errors += 1; continue

        inputs = vproc(frames, return_tensors='pt')
        inputs = {k: v.to(device)
```

```
                    for k, v in inputs.items()}

        with torch.no_grad(),
             torch.amp.autocast('cuda'):
            out = vmodel(**inputs)
            feat = out.last_hidden_state.mean(dim=1)
                   .cpu().float().squeeze(0)

        features.append(feat)
        labels.append(clip['label'])
```

Explanation:

- **Main extraction loop** processes each video clip starting from `start_idx`.
- `load_frames_from_file` extracts 8 uniformly-spaced frames from the video.
- `vproc(frames, return_tensors='pt')` — The video processor normalizes pixel values, applies model-specific preprocessing, and returns PyTorch tensors.
- `inputs = {k: v.to(device) ...}` — Moves all input tensors to GPU.
- `torch.no_grad()` — Disables gradient computation (saves memory, since we are only doing inference).
- `torch.amp.autocast('cuda')` — Enables automatic mixed precision; operations run in FP16 where safe, FP32 where needed.
- `vmodel(**inputs)` — Forward pass through V-JEPA 2.
- `out.last_hidden_state` — The final layer's hidden state tensor of shape `[1, num_patches, 1280]`.
- `.mean(dim=1)` — Mean-pools across all spatial/temporal patches, producing a single 1280-dimensional vector per video. This is a standard technique for converting patch-level representations to a video-level representation.
- `.cpu().float().squeeze(0)` — Moves to CPU, converts to FP32, and removes the batch dimension.

```
        if (i+1) % CKPT_EVERY == 0:
            torch.save({
                'features': torch.stack(features),
                'labels': torch.tensor(labels),
                'next_idx': i+1}, VJEPA_CKPT)
```

Explanation:

- Every 100 clips, a checkpoint is saved containing all features extracted so far, their labels, and the next index to process.
- `torch.stack(features)` — Combines the list of 1280-dim vectors into a single tensor of shape `[N, 1280]`.
- This enables resuming from the last checkpoint if the Kaggle kernel crashes or times out.

After the loop completes, the final features are saved and the model is unloaded from GPU memory.

Output: Feature tensor shape: `torch.Size([172, 1280])` — 172 video clips, each represented by a 1280-dimensional vector.

Cell 6: Qwen2.5-VL Feature Extraction

```
from transformers import
    Qwen2_5_VLForConditionalGeneration,
    AutoProcessor, BitsAndBytesConfig

bnb = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_quant_type='nf4',
    bnb_4bit_use_double_quant=True)

qmodel = Qwen2_5_VLForConditionalGeneration
    .from_pretrained(
    'Qwen/Qwen2.5-VL-7B-Instruct',
    quantization_config=bnb,
    device_map='auto',
    torch_dtype=torch.float16)
qproc = AutoProcessor.from_pretrained(
    'Qwen/Qwen2.5-VL-7B-Instruct')
qmodel.eval()
```

Explanation:

- `Qwen2_5_VLForConditionalGeneration` — The Qwen2.5-VL 7B vision-language model. It is an autoregressive model that processes both visual and textual inputs.
- `AutoProcessor` — Automatically selects the correct image processor and tokenizer for the model.
- `BitsAndBytesConfig` — Configures 4-bit quantization to fit the 7B-parameter model into limited GPU memory:
  - `load_in_4bit=True` — Quantizes weights to 4-bit precision (∼3.5 GB instead of ∼14 GB for FP16).
  - `bnb_4bit_compute_dtype=torch.float16` — Computations are performed in FP16 (dequantized on-the-fly).
  - `bnb_4bit_quant_type='nf4'` — NormalFloat4 quantization; optimal for normally-distributed weights.
  - `bnb_4bit_use_double_quant=True` — Quantizes the quantization constants themselves, saving additional memory.
- `device_map='auto'` — Automatically distributes model layers across available

GPUs (and CPU if needed).

```
    frames = load_frames_from_file(clip['path'], 4,
                                   IMG_SIZE)
    messages = [{'role': 'user', 'content': [
        {'type': 'video', 'video': frames},
        {'type': 'text', 'text': 'Describe.'},
    ]}]
    text = qproc.apply_chat_template(messages,
        tokenize=False, add_generation_prompt=True)
    inputs = qproc(text=[text], videos=[frames],
        return_tensors='pt', padding=True)

    with torch.no_grad():
        out = qmodel(**inputs,
                     output_hidden_states=True)
        feat = out.hidden_states[-1].mean(dim=1)
               .cpu().float().squeeze(0)
```

Explanation:

- **4 frames** are used instead of 8 (Qwen processes more tokens per frame, so fewer frames save GPU memory).
- `messages` — Structured as a chat-format input with a video and a text prompt ("Describe.").
- `apply_chat_template` — Converts the message list into the model's expected text format with special tokens (e.g., `<|im_start|>user ... <|im_end|>`).
- `qproc(text=..., videos=...)` — Processes both text tokens and video frames into model-ready tensors (pixel values, input IDs, attention masks).
- `output_hidden_states=True` — Tells the model to return hidden states from all transformer layers (not just the final logits).
- `out.hidden_states[-1]` — Takes the last layer's hidden states; shape `[1, seq_len, 3584]`.
- `.mean(dim=1)` — Mean-pools across the sequence dimension, producing a 3584-dimensional video representation.
- The 3584-dim feature vector captures the model's understanding of the video content.

Output: Feature tensor shape: `torch.Size([172, 3584])` — 172 clips, each represented by a 3584-dimensional vector.

Cell 7: Load Features and Create Train/Eval Split

```
vdata = torch.load(VJEPA_FEAT, map_location='cpu',
                   weights_only=True)
qdata = torch.load(QWEN_FEAT, map_location='cpu',
                   weights_only=True)

vf = vdata['features'].numpy()
qf = qdata['features'].numpy()
vl = vdata['labels'].numpy()
ql = qdata['labels'].numpy()
n = min(len(vl), len(ql))
vf, qf, labels = vf[:n], qf[:n], vl[:n]
```

Explanation:

- Loads both saved feature files into CPU memory.
- `.numpy()` — Converts PyTorch tensors to NumPy arrays (required by scikit-learn).
- `n = min(len(vl), len(ql))` — Ensures both models have the same number of samples (172).
- Arrays are truncated to `n` samples for fair comparison.

```
n_tr = int(n * TRAIN_SPLIT)
perm = np.random.RandomState(SEED).permutation(n)
tr, ev = perm[:n_tr], perm[n_tr:]
y_tr, y_ev = labels[tr], labels[ev]
```

Explanation:

- `n_tr = int(172 * 0.8) = 137` — 137 training samples, 35 evaluation samples.
- `np.random.RandomState(SEED).permutation(n)` — Creates a deterministic random permutation of indices 0 to 171. Using a `RandomState` object (not the global seed) ensures the same split regardless of prior random operations.
- `tr`, `ev` — Index arrays for training and evaluation sets.
- `y_tr`, `y_ev` — Corresponding label arrays.

Output: Train: 137 — Eval: 35. Per-class distribution: BB: 38/9, FX: 34/11, UB: 38/9, VT: 27/6.

Cell 8: Run Classifiers and Compare

```
def run_classifiers(X_tr, y_tr, X_ev, y_ev):
    scaler = StandardScaler()
    X_tr_s = scaler.fit_transform(X_tr)
    X_ev_s = scaler.transform(X_ev)
    pca = PCA(n_components=0.95, random_state=SEED)
    X_tr_p = pca.fit_transform(X_tr_s)
    X_ev_p = pca.transform(X_ev_s)
```

Explanation:

- **StandardScaler:** Standardizes each feature to zero mean and unit variance. This is critical because classifiers like SVM and KNN are sensitive to feature scales. Formula: $z_j = \frac{x_j - \mu_j}{\sigma_j}$.
- `fit_transform(X_tr)` — Computes μ and σ from training data and transforms it.
- `transform(X_ev)` — Transforms evaluation data using the *same* μ and σ (no data leakage).
- **PCA:** Reduces dimensionality while retaining 95% of the total variance.
  - V-JEPA: $1280 \rightarrow 70$ dimensions.
  - Qwen: $3584 \rightarrow 97$ dimensions.
- PCA reduces noise and computational cost while preserving the most informative feature directions.

```
    classifiers = {
        'SVM-RBF': SVC(kernel='rbf', C=10,
            gamma='scale',
            class_weight='balanced',
            random_state=SEED, probability=True),
        'KNN-7': KNeighborsClassifier(n_neighbors=7,
            n_jobs=-1),
        'RandomForest': RandomForestClassifier(
            n_estimators=500,
            class_weight='balanced',
            random_state=SEED, n_jobs=-1),
        'MLP': MLPClassifier(
            hidden_layer_sizes=(512, 256),
            max_iter=1000, alpha=1e-4,
            random_state=SEED,
            early_stopping=True,
```

```
            learning_rate='adaptive'),
        'LogReg': LogisticRegression(C=1.0,
            class_weight='balanced',
            max_iter=1000, random_state=SEED,
            multi_class='multinomial'),
    }
```

Explanation:

Five classifiers are configured:

1. **SVM-RBF:** Support Vector Machine with Radial Basis Function kernel. `C=10` controls regularization (higher → tighter fit). `gamma='scale'` sets $\gamma = \frac{1}{d \cdot \text{Var}(X)}$. `class_weight='balanced'` adjusts weights inversely proportional to class frequencies.
2. **KNN-7:** k-Nearest Neighbors with $k = 7$. Classifies based on majority vote of 7 nearest training points in feature space. `n_jobs=-1` uses all CPU cores.
3. **Random Forest:** Ensemble of 500 decision trees using bootstrap sampling and random feature subsets. Each tree votes; majority wins. Robust to overfitting.
4. **MLP:** Two hidden layers (512 and 256 neurons). `alpha=1e-4` is L2 regularization strength. `early_stopping=True` stops training when validation loss plateaus. `learning_rate='adaptive'` reduces the learning rate when loss stops decreasing.
5. **Logistic Regression:** Linear classifier with softmax for multi-class. `C=1.0` is inverse regularization strength. `multi_class='multinomial'` uses the cross-entropy loss across all classes simultaneously.

The function then trains each classifier, computes predictions, and calculates accuracy, F1-score, and MCC. The best classifier (by F1) is selected, and a full classification report with confusion matrix is generated.

Key Results:

- **V-JEPA 2:** Best classifier = LogReg, Accuracy = 97.1%, F1 = 96.4%, MCC = 0.962.
- **Qwen2.5-VL:** Best classifier = LogReg, Accuracy = 80.0%, F1 = 79.7%, MCC = 0.731.

Cell 9: Final Comparison Table

```
print('FINAL COMPARISON -- FineGym (4 Event Classes)')

def row(name, v1, v2, pct=True):
    if pct:
        print(f'  {name:<25} {v1*100:>17.1f}%
                {v2*100:>17.1f}%')
    else:
        print(f'  {name:<25} {v1:>18.3f}
                {v2:>18.3f}')

row('Accuracy', vj_m['accuracy'], qw_m['accuracy'])
row('Precision (macro)', ...)
row('F1 (macro)', ...)
row('MCC', ..., pct=False)
row("Cohen's Kappa", ..., pct=False)
```

Explanation:

- The `row()` helper function prints a formatted comparison row, optionally converting values to percentages.
- All metrics from both models are displayed side-by-side for easy comparison.
- Architecture details (parameter count, feature dimension, paradigm, modality) are also printed.

```
winner = 'V-JEPA 2' if vj_f1 > qw_f1 else 'Qwen2.5-VL'
margin = abs(vj_f1 - qw_f1) * 100
print(f'WINNER: {winner} (by {margin:.1f}% F1)')
```

Explanation:

- Compares the macro F1-scores of both models.
- Declares V-JEPA 2 as the winner with a 16.7% F1 margin.
- Confusion matrices are printed for both models showing per-class classification patterns.
- Final results are saved as a JSON file for further analysis.

Final Output Summary:

Metric	**V-JEPA 2 (654M)**	**Qwen2.5-VL (7B)**
Accuracy	97.1%	80.0%
Precision (macro)	97.5%	80.6%
Recall (macro)	95.8%	79.7%
F1 (macro)	96.4%	79.7%
MCC	0.962	0.731
Cohen's Kappa	0.961	0.729
Feature dim	1280	3584
Best classifier	LogReg	LogReg

Table 1: FineGym 4-Event Classification Results

Winner: V-JEPA 2 (by 16.7% F1)